



S.P.M College, Udantpuri

Bachelor Of Computer Application (BCA)

Part – 2 (Paper-4)

– Hira Kumar

❖ Function Overloading

- Function overloading means defining multiple functions with the same name but with different parameters (number of parameters or data_type).
- It allows functions to perform similar tasks with different types or numbers of arguments.
- It is a part of compile time polymorphism.

Poly means → Many & Morph means → forms
(Several forms of the same name)

WAP to use function overloading.

```
#include<iostream>
#include<conio.h>
using namespace std;
void hira(int a, int b) → first function
{
    cout << " sum of two number is = " << a+b << endl;
}
void hira(int a) → second function
{
    if (a%2 == 0)
        cout << "a is even";
    else
        cout << "a is odd";
}
int main()
{
    hira(10,30); → argument different
    hira (10);
    getch();
}
```

parameters same (int a)

argument different

```
J:\c-c++\part-3\2.exe
sum of two number is = 40
a is even
```

Questions

HIRA KUMAR

1. Write a C++ program to demonstrate(दिखाना) function overloading by creating three functions named add() to add:
Two integers, Two floating-point numbers, Three integers.
2. Write a C++ program to overload the area() function to calculate:
The area of a square
The area of a rectangle
The area of a circle
3. Write a c++ program to demonstrate function overloading by creating two functions named display().
One that takes an integer argument & one that takes a string argument.
4. Write a C++ program to overload a function calculate() to find:
The square of an integer
The cube of an integer
The square of a floating-point number

5. Discount Calculation

Write a C++ program to overload a function discount() to calculate the discount based on:

- A fixed discount amount
- A percentage discount
- A percentage discount with an additional membership discount

=====

Hint :

Input:

Price = 1000, Fixed discount = 100

Price = 1000, Discount = 10%

Price = 1000, Discount = 10%, Membership discount = 50

Expected Output:

Final Price = 900

Final Price = 900

Final Price = 850

=====

➤ Call by value

- Call by Value is a method of passing arguments to a function in which a copy of the actual value is passed to the function.
- Changes made to the parameter do not affect the original variable.
- The function works with a separate copy i.e. Photo copy of the data.

Write a program in C++ in which use call by value.

```
#include<iostream>
#include<conio.h>
using namespace std;

void hira(int a, int b) //formal parameters
{
    a*=10;
    b+=10;
    cout << "Value of a =" << a << endl;
    cout << "Value of b =" << b << endl;
}

int main()
{
    cout << " Use Call by value" << endl;
    int x=5,y=10;
    hira (x,y); // actual parameters
    cout << " ===== " << endl;
    cout << "Value of x = " << x << endl;
    cout << "value of y = " << y << endl;

    getch();
    return 0;
}
```

C:\Users\hira\Desktop\hira.exe

```
Use Call by value
Value of a =50
Value of b =20
=====
Value of x = 5
value of y = 10
```

OR

```
#include<iostream>
#include<conio.h>
using namespace std;

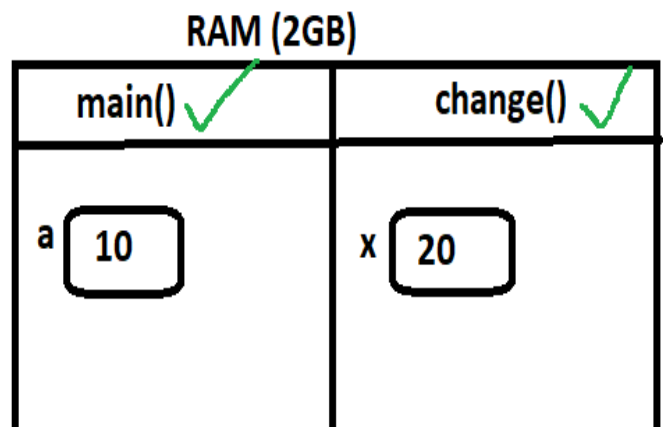
void change(int x)
{
    x = 20;
}

int main()
{
    int a = 10;
    change(a);
    cout << "a = " << a;
    getch();

    return 0;
}
```

C:\Users\hira\Desktop\hello.exe

a = 10



➤ What is Address ?

In C++, an address is the location of a variable in computer memory (RAM) where its value is stored.

```
#include<iostream>
#include<conio.h>
using namespace std;

int main()
{
    int a= 10;
    cout << "Value of a = " << a << endl;
    cout << "Address of a = " << &a << endl;
    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
Value of a = 10
Address of a = 0x5dbc9ff86c
```

RAM (2GB)



❖ Reference Variable

- A reference variable is another name (alias) for an existing variable. It does not create a separate variable or separate memory location.
- Syntax : `data_type &reference_name = original_variable;`
- Example :

```
Int a=10;
```

```
int &hira = a; // reference variable
```

output : a=10, hira=10;

➤ Call by reference

- Call by Reference is a method of passing arguments to a function in which the reference (address) of the original variable is passed to the function.
- The function works with the original variable.
- Changes made to the parameter affect the original variable.
- In C++, it is commonly done using the reference (&) operator.
- The original value can be modified.
- **No separate memory is allocated for the user-defined function.**

Write a program in C++ in which use call by reference.

```
#include<iostream>
#include<conio.h>
using namespace std;

void hira(int &a, int b) //formal parameters
{
    a*=10;
    b+=10;
    cout << "Value of a =" << a << endl;
    cout << "Value of b =" << b << endl;
}

int main()
{
    cout << " Use Call by Reference)" << endl;
    int x=5,y=10;
    hira (x,y); // actual parameters
    cout << " ======" << endl;
    cout << "Value of x = " << x << endl;
    cout << "value of y = " << y << endl;

    getch();
    return 0;
}
```

```
C:\Users\hira\Desktop\hira.exe

Use Call by Reference)
Value of a =50
Value of b =20
=====
Value of x = 50
value of y = 10
```

OR

```
#include<iostream>
#include<conio.h>
using namespace std;

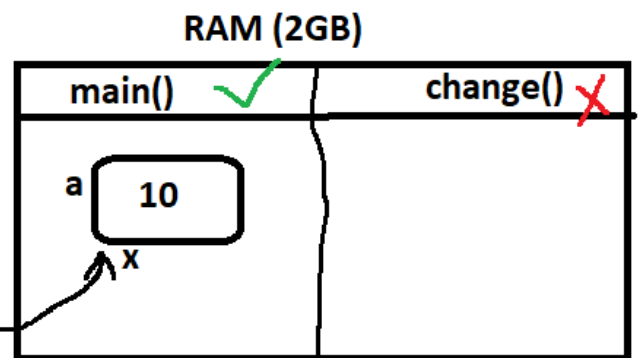
void change(int &x)
{
    x = 20;
}

int main()
{
    int a = 10;
    change(a);
    cout << "a = " << a;
    getch();

    return 0;
}
```

```
C:\Users\hira\Desktop\hello.exe

a = 20
```



➤ What is pointer ?

- A pointer is a variable that stores the memory address of another variable.
- **Syntax :** dataType *pointerName;
- **Example :** int *p; , int *hira;

```

#include<iostream>
#include<conio.h>
using namespace std;

int main()
{
    int a= 10;
    int *p;
    cout << "Value of a = " << a << endl;
    cout << "Address of a = " << &a << endl;
    p=&a;
    cout << " value of p = " << p << endl;
    cout << " Value of *p = " << *p << endl;
    *p=20;
    cout << " value of a = " << a << endl;
    getch();
}

```

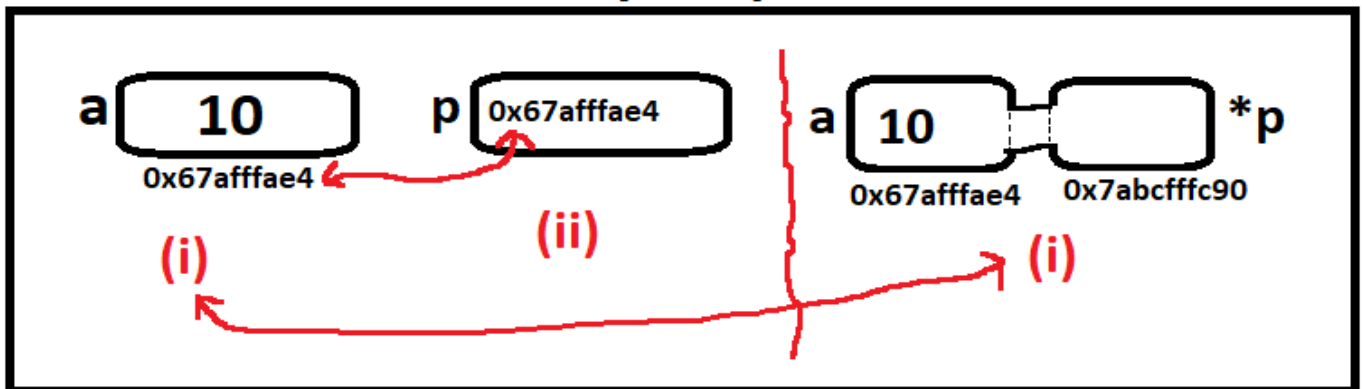
C:\Users\hira\Desktop\hello.exe

```

Value of a = 10
Address of a = 0x67afffae4
value of p = 0x67afffae4
Value of *p = 10
value of a = 20

```

RAM (2GB)



Here:

- $a \rightarrow$ normal variable that stores **10**
- $\&a \rightarrow$ address of a
- $p \rightarrow$ pointer variable that stores the **address of a**
- $*p \rightarrow$ value stored at that address, i.e. **10**

➤ Call by Pointer

- Call by Pointer is a method of passing arguments to a function in which the address of the original variable is passed using a pointer. (variable के address को पॉइंटर के मदद से function के पास भेजा जाता हैं)
- The function receives the address of the original variable.
- The pointer is used with the dereference operator $*$ to access or modify the original value.
- Changes made through the pointer affect the original variable.

Write a program in C++ in which use call by pointer.

```
#include<iostream>
#include<conio.h>
using namespace std;

void change(int *x)
{
    *x = 20;
}

int main()
{
    int a = 10;
    change(&a);
    cout << "a = " << a;
    getch();
    return 0;
}
```

C:\Users\hira\Desktop\hello.exe

a = 20

```
#include<iostream>
#include<conio.h>
using namespace std;

void hira(int *a, int b) //formal parameters
{
    *a*=10;
    b+=10;
    cout << "Value of a =" << *a << endl;
    cout << "Value of b =" << b << endl;
}

int main()
{
    cout << " Use Call by pointer)" << endl;
    int x=5,y=10;
    hira (&x,y); // actual parameters
    cout << " =====" << endl;
    cout << "Value of x = " << x << endl;
    cout << "value of y = " << y << endl;

    getch();
    return 0;
}
```

C:\Users\hira\Desktop\hira.exe

Use Call by pointer)

Value of a =50

Value of b =20

=====

Value of x = 50

value of y = 10

Questions (Find output)

1. #include <iostream>
using namespace std;

```
void change(int x)
{
    x = 20;
}
```

```
int main()
{
    int a = 10;
    change(a);
    cout << a;
}
```

Find the output.

2.

```
#include <iostream>
using namespace std;
```

```
void add(int a, int b)
{
    a = a + b;
    cout << a;
}
```

```
int main()
{
    int x = 10, y = 20;
    add(x, y);
    cout << " " << x;
}
```

Find the output.

3.

```
#include <iostream>
using namespace std;
```

```
void change(int &x)
{
    x = x + 10;
}
```

```
int main()
{
    int a = 10;
    change(a);
    cout << a;
}
```

Find the output.

4.

```
#include <iostream>
using namespace std;
```

```
void swapValue(int &a, int &b)
{
    int t = a;
    a = b;
    b = t;
}
```

```
int main()
```

```
{
    int x = 10, y = 20;
    swapValue(x, y);
    cout << x << " " << y;
}
```

Find the output.

5.

```
#include <iostream>
using namespace std;
```

```
void change(int *p)
{
    *p = 50;
}
```

```
int main()
{
    int a = 10;
    change(&a);
    cout << a;
}
```

Find the output.

6.

```
#include <iostream>
using namespace std;
```

```
void increment(int *p)
{
    *p = *p + 5;
}
```

```
int main()
{
    int a = 10;
    increment(&a);
    cout << a;
}
```

Find the output.

7.

```
#include <iostream>
using namespace std;
```

```
void value(int x)
{
```

```

    x = 20;
}

void reference(int &x)
{
    x = 30;
}

int main()
{
    int a = 10;

    value(a);
    cout << a << " ";

    reference(a);
    cout << a;
}

```

Find the output.

8.

```

#include <iostream>
using namespace std;

```

```

void fun1(int x)
{
    x = 20;
}

```

```

void fun2(int *x)
{
    *x = 30;
}

```

```

int main()
{
    int a = 10;

    fun1(a);
    cout << a << " ";

    fun2(&a);
    cout << a;
}

```

Find the output.

9.

```
#include <iostream>
using namespace std;

void change(int &a, int *p)
{
    a = a + 5;
    *p = *p + 10;
}

int main()
{
    int x = 10, y = 20;

    change(x, &y);

    cout << x << " " << y;
}
```

Find the output.

10.

```
#include <iostream>
using namespace std;

void fun(int a, int &b, int *c)
{
    a = a + 10;
    b = b + 20;
    *c = *c + 30;
}

int main()
{
    int x = 5, y = 10, z = 15;

    fun(x, y, &z);

    cout << x << " " << y << " " << z;
}
```

Find the output.

Answers – (Match)

Question 5: 10

Question 4: 30 10

Question 3: 20

Question 2: 20 10

Question 1: 50

Question 10: 15

Question 9: 10 30

Question 8: 10 30
Question 7: 15 30
Question 6: 5 30 45

=====